

CIVILIZATIONS



Autores: Jairo, Izan y Diego

Civilizations	3
1. Introducción del proyecto	3
Descripción del juego	3
Objetivos del proyecto	3
Herramientas utilizadas	4
2. Análisis y diseño	4
Especificaciones funcionales	4
Especificaciones no funcionales	5
Estructura del proyecto	5
Diseño de mecánicas	5
Diagrama de clases	6
Diagrama de secuencia simplificado de una batalla	7
3. Arquitectura y desarrollo	8
Estructura del código fuente	8
Componentes principales	8
Flujo de datos y eventos	9
4. Test y depuración	10
Pruebas realizadas	10
Errores encontrados y soluciones	10
5. Recursos gráficos y de audio	11
6. Manual de usuario	11
Cómo ejecutar el juego	11
Objetivo del juego	11
Controles	11
Descripción de las opciones del menú	12
Ejemplo de una partida	12
7. Reflexión y mejoras	13
Dificultades encontradas	13
Cómo se superaron	13
Mejoras futuras sugeridas en el PDF	14

1. Introducción del proyecto

Descripción del juego

Civilizations es un juego de estrategia en consola donde el jugador gestiona una civilización medieval. Debe administrar recursos (comida, madera, hierro y maná), construir edificios que mejoran la generación de recursos, investigar tecnologías de ataque y defensa, y reclutar un ejército con unidades de infantería, defensas fijas y unidades especiales. Periódicamente, un ejército enemigo ataca la civilización y se desencadena una batalla táctica automática. El objetivo es sobrevivir el mayor número de batallas, optimizando la economía y la composición del ejército.

Objetivos del proyecto

- Aplicar los conceptos de programación orientada a objetos (herencia, polimorfismo, clases abstractas, interfaces).
- Desarrollar una lógica de batalla compleja con probabilidades, recursos y persistencia en base de datos.
- Construir una arquitectura modular que separe claramente las responsabilidades (juego, batalla, tareas programadas, interfaz web).
- Documentar profesionalmente el proceso de desarrollo, desde el diseño hasta el manual de usuario.

Herramientas utilizadas

Componente	Tecnología
Backend	Java, compilado y ejecutado en terminal(VS Code)
Base de datos	SQL (script <code>create_tables.sql</code>)
Frontend web	Node.js + hbs + CSS
Control de versiones	Git + GitHub
Diagramas	Mermaid Live Editor
Documentación	Drive , PDF

2. Análisis y diseño

Especificaciones funcionales

- El jugador puede construir edificios (granja, carpintería, herrería, torre mágica, iglesia) que incrementan la generación de recursos.
- Puede mejorar las tecnologías de ataque y defensa, cuyo coste aumenta con cada nivel.
- Puede reclutar 9 tipos de unidades (Swordsman, Spearman, Crossbow, Cannon, ArrowTower, Catapult, RocketLauncherTower, Magician, Priest).
- Un ejército enemigo se genera automáticamente cada 3 minutos, usando recursos crecientes según el número de batallas ya libradas.
- Las batallas se resuelven siguiendo un algoritmo detallado con selección probabilística de atacantes y defensores, daños, residuos y cambio de turno.
- Al finalizar cada batalla se almacena un informe completo y se actualizan los recursos de la civilización.

Especificaciones no funcionales

- Rendimiento: Las batallas deben ejecutarse en tiempo real sin bloquear el menú de la consola (se usan `TimerTask`).
- Modularidad: El código está organizado en 22 clases, cada una con una responsabilidad bien definida.
- Portabilidad: Compatible con cualquier sistema que soporte Java 8 y Oracle.
- Mantenibilidad: Las constantes del juego están centralizadas en la interfaz `Variables`.

Estructura del proyecto

La aplicación se compone de tres bloques principales:

1. Juego en consola: Clases Java que modelan la civilización, las batallas y el menú interactivo.
2. Base de datos: Esquema Oracle que almacena el estado completo de la civilización y el historial de batallas.
3. Aplicación web (en desarrollo): Servidor Node.js con vistas EJS que muestra los datos de la base de datos.

Diseño de mecánicas

Recursos: Comida, madera, hierro y maná se generan cada minuto. Los edificios aumentan la producción base. El maná solo se produce si hay torres mágicas.

Edificios: Cada tipo proporciona un bono específico (más comida, madera, hierro o maná). Su construcción consume recursos y está limitada por las excepciones `ResourceException`.

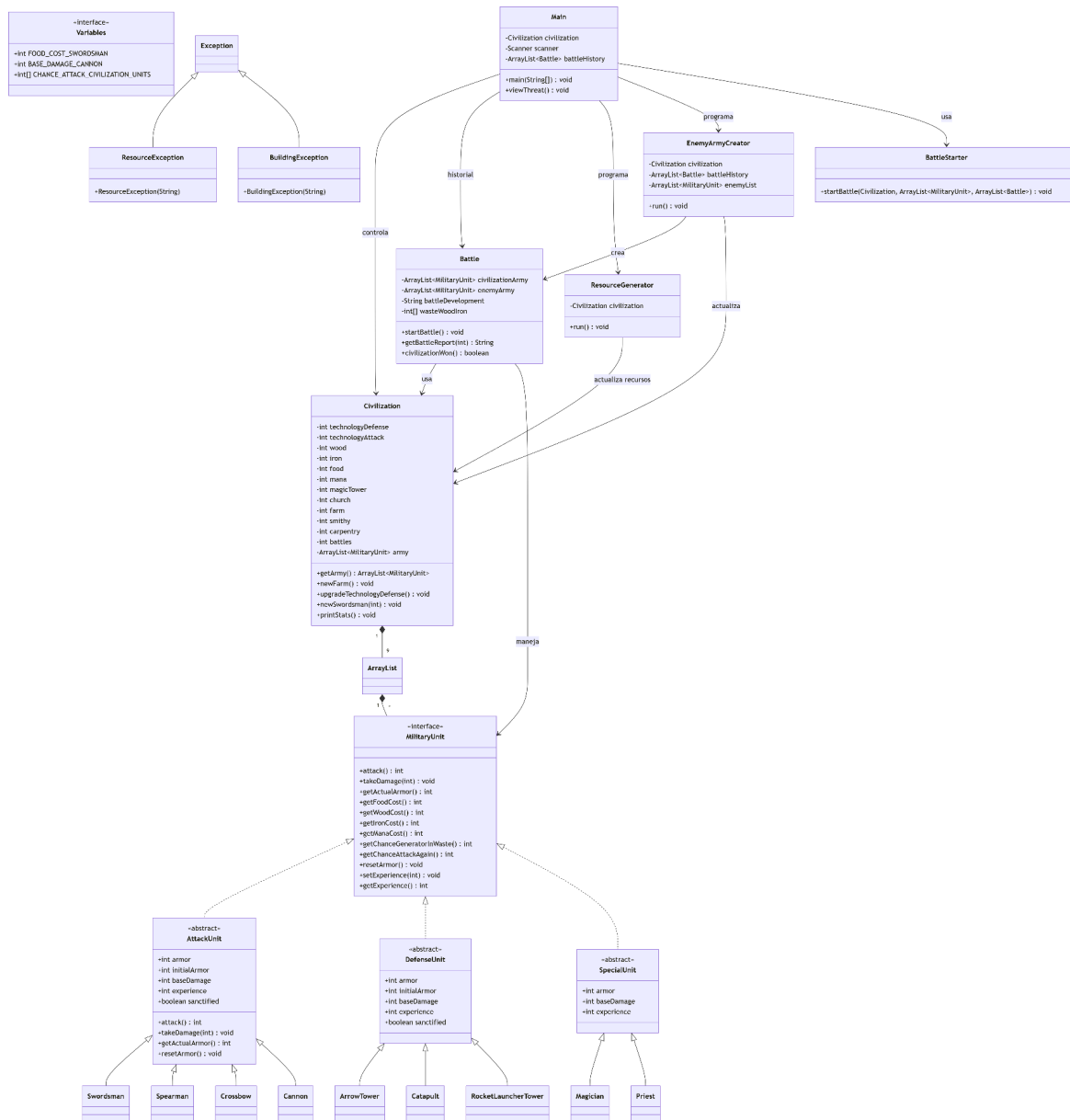
Tecnologías: Mejoran las estadísticas de las unidades creadas posteriormente. No afectan a las unidades ya existentes.

Unidades: Se dividen en tres categorías (ataque, defensa, especiales) que heredan de clases abstractas. Cada unidad conoce sus costes y sus probabilidades de generar residuos o atacar de nuevo. La experiencia y la santificación (otorgada por sacerdotes) aumentan el daño y la armadura.

Batalla: Sigue un algoritmo probabilístico. Cada turno, un bando ataca según las probabilidades definidas en `Variables`. El defensor se elige proporcionalmente al número de unidades de cada tipo. Si una unidad es eliminada, puede generar residuos (madera y hierro). El combate continúa hasta que un bando pierde el 80% de sus unidades iniciales. El ganador se determina por las pérdidas ponderadas (hierro + madera/5 + comida/10).

Diagrama de clases

El siguiente diagrama muestra las relaciones entre las interfaces, clases abstractas y clases concretas del proyecto. Se ha omitido la lista exhaustiva de atributos y métodos por claridad, representando solo los más significativos.



Explicación del diagrama

- Interfaces: `Variables` agrupa las constantes; `MilitaryUnit` define el contrato que deben cumplir todas las unidades.
- Clases abstractas: `AttackUnit`, `DefenseUnit` y `SpecialUnit` implementan la lógica común de ataque y defensa, y delegan en las subclasses los detalles concretos (costes, probabilidades).
- Unidades concretas: Nueve clases que heredan de las abstractas y devuelven los valores definidos en `Variables`.
- Excepciones: `ResourceException` y `BuildingException` se lanzan cuando no hay recursos suficientes o faltan edificios requeridos.
- Civilization: Contiene toda la información del estado del juego y los métodos para modificarlo.
- Battle: Orquesta el combate entre dos ejércitos y genera informes.
- Main: Punto de entrada, menú interactivo y lanzamiento de tareas programadas.
- Tareas programadas: `ResourceGenerator` (cada minuto), `EnemyArmyCreator` (cada 3 minutos) y `BattleStarter` (inicia la batalla).

Diagrama de secuencia simplificado de una batalla

1. `EnemyArmyCreator.run()` crea el ejército enemigo.
 2. `EnemyArmyCreator` llama a `BattleStarter.startBattle()`.
 3. `Battle.startBattle()` ejecuta el bucle de combate:
 - Elegir bando atacante aleatoriamente.
 - Elegir grupo atacante según `CHANCE_ATTACK*_UNITS`.
 - Elegir grupo defensor según proporción de unidades.
 - Calcular daño y aplicar a la unidad defensora.
 - Si muere, generar posibles residuos y eliminar la unidad.
 - Comprobar si el atacante repite turno.
 - Cambiar turno si no repite.
 4. Al terminar, `Battle` calcula las pérdidas ponderadas y determina el ganador.
 5. `EnemyArmyCreator` actualiza los recursos de `Civilization` con los residuos y el contador de batallas.
 6. `Main` almacena el objeto `Battle` en el historial para su consulta.
-

3. Arquitectura y desarrollo

Estructura del código fuente

El proyecto se organiza en un único paquete `civilizations` con 22 archivos:

```
M03_Programacion/  
└─ civilizations/  
    │ Variables.java  
    │ MilitaryUnit.java  
    │ ResourceException.java  
    │ BuildingException.java  
    │ AttackUnit.java  
    │ DefenseUnit.java  
    │ SpecialUnit.java  
    │ Swordsman.java  
    │ Spearman.java  
    │ Crossbow.java  
    │ Cannon.java  
    │ ArrowTower.java  
    │ Catapult.java  
    │ RocketLauncherTower.java  
    │ Magician.java  
    │ Priest.java  
    │ Civilization.java  
    │ Battle.java  
    │ Main.java  
    │ ResourceGenerator.java  
    │ EnemyArmyCreator.java  
  
    └─ BattleStarter.java
```

Componentes principales

Civilization.java

- Atributos privados para recursos, edificios, tecnologías y el ejército (un `ArrayList` de 9 `ArrayList<MilitaryUnit>`).
- Métodos públicos para construir edificios (`newFarm()`, etc.) y mejorar tecnologías (`upgradeTechnologyDefense()`).
- Métodos para añadir unidades (`newSwordsman(int n)`, etc.), que validan recursos y edificios, lanzando las excepciones pertinentes.
- `createUnitByType(int type)`: Calcula la armadura y el daño según los niveles de tecnología actuales y devuelve una nueva unidad.
- `printStats()`: Muestra el estado completo de la civilización formateado.

Battle.java

- Constructor que recibe el ejército de la civilización (como array de 9 grupos) y la lista enemiga. Calcula los costes iniciales y los conteos.

- `startBattle()`: Bucle principal del combate.
- Métodos privados para seleccionar grupos atacantes/defensores según las probabilidades definidas en `Variables`.
- `calculateResults()`: Calcula las pérdidas ponderadas y determina el ganador.
- `getBattleReport(int)` y `getBattleDevelopment()`: Generan los informes detallados.

Main.java

- Contiene el método `main`, que instancia `Civilization`, `Scanner` y las tareas programadas.
- Menú interactivo con `do-while` y `switch` que permite acceder a todas las funcionalidades.
- Métodos estáticos para cada opción del menú: `buildMenu()`, `upgradeMenu()`, `createUnitsMenu()`, `viewThreat()`, `viewBattleHistory()`.

Tareas programadas

- `ResourceGenerator`: Extiende `TimerTask`. Cada minuto añade los recursos base más los bonus por edificios.
- `EnemyArmyCreator`: Extiende `TimerTask`. Cada 3 minutos crea un ejército enemigo con recursos crecientes y lanza la batalla automáticamente.
- `BattleStarter`: Clase de utilidad con un método estático para iniciar una batalla y actualizar la civilización.

Flujo de datos y eventos

1. Al iniciar el programa, se crea una `Civilization` con valores por defecto (todo a cero, ejército vacío).
 2. `ResourceGenerator` añade recursos cada minuto (incluso mientras el jugador está en el menú).
 3. El jugador puede construir edificios, mejorar tecnologías y reclutar unidades.
 4. Cada 3 minutos, `EnemyArmyCreator` genera un ejército enemigo y lanza una batalla.
 5. Durante la batalla, se consumen unidades y se generan residuos. Si la civilización gana, los residuos se añaden a los recursos.
 6. El resultado de cada batalla se guarda en el `ArrayList<Battle> battleHistory` de `Main`.
 7. El jugador puede consultar el historial de batallas y ver informes detallados.
-

4. Test y depuración

Pruebas realizadas

- Compilación limpia de todas las clases: Se verificó que no hubiera errores de sintaxis ni dependencias circulares.
- Ejecución del menú: Se probaron una a una las 7 opciones del menú principal, confirmando que responden correctamente incluso con entradas no válidas (letras donde se esperan números).
- Construcción de edificios y tecnologías: Se comprobó que las excepciones `ResourceException` se lanzan cuando no hay recursos suficientes y que el mensaje informativo es claro.
- Reclutamiento de unidades: Se testearon los 9 tipos de unidades, verificando que se crean con las estadísticas correctas (incluyendo bonus por tecnología) y que se lanzan `ResourceException` y `BuildingException` cuando corresponde.
- Batallas: Se simularon varias batallas con diferentes tamaños de ejército para asegurar que el bucle termina correctamente, que los residuos se calculan bien y que el ganador se determina según la fórmula ponderada.
- Persistencia: Se ejecutó el script SQL en Oracle y se comprobó que las tablas se crean con las restricciones adecuadas.

Errores encontrados y soluciones

1. `ResourceException` al crear unidades sin recursos
Síntoma: Al pedir 10 Swordsman sin comida suficiente, el programa lanzaba la excepción pero no informaba cuántas unidades se habían creado realmente.
Solución: Se modificó el método `addUnits()` para que, tras el bucle, compare `created` con `n` y lance la excepción con el mensaje "Solo se pudieron añadir X de N unidades".
2. Problema con el conteo de unidades enemigas en `Battle`
Síntoma: Los Swordsman enemigos no se contabilizaban en `actualNumberUnitsEnemy[0]` porque se usaba la variable `enemyDrops` por error.
Solución: Se corrigió el bucle de inicialización para que cada tipo de unidad enemiga incremente su contador correspondiente.
3. Determinación incorrecta del ganador en el informe
Síntoma: `getBattleReport()` siempre mostraba que ganaba la civilización porque se evaluaba una condición basada en el texto del log, no en los datos reales.
Solución: Se modificó el método para que compare `resourcesLooses[0][3]` y `resourcesLooses[1][3]` (pérdidas ponderadas) y muestre el ganador correcto.
4. Aviso de `unchecked cast` en la declaración del ejército
Síntoma: La línea `army = (ArrayList<MilitaryUnit>[]) new ArrayList[9]` generaba un warning.
Solución: Se cambió la estructura interna a `ArrayList<ArrayList<MilitaryUnit>>` y se añadió un método `getArmy()` que devuelve el array necesario para `Battle`.

5. Recursos gráficos y de audio

- Imágenes: En la versión actual (consola) no se utilizan recursos gráficos. La aplicación web (M04) emplea imágenes de placeholder para los iconos de recursos (madera, hierro, etc.) y logotipo del juego. Estas imágenes se han tomado de repositorios libres o se han generado con herramientas de diseño básicas.
 - Audio: No se ha implementado sonido en ninguna de las versiones.
 - Fuentes: Se ha empleado la fuente Georgia (serif) para la interfaz web, con el objetivo de evocar un ambiente medieval.
-

6. Manual de usuario

Cómo ejecutar el juego

1. Asegúrate de tener Java 8 o superior instalado.
2. Descarga el proyecto desde el repositorio GitHub.
3. Abre una terminal en la carpeta M03_Programacion/civilizations/.
4. Compila: `javac Main.java`
5. Ejecuta: `java Main`

Objetivo del juego

Administra tu civilización para resistir el mayor número posible de ataques enemigos. Construye edificios que aumenten tus recursos, investiga tecnologías para mejorar tus tropas y recluta un ejército equilibrado.

Controles

El juego se maneja mediante un menú numérico. Simplemente introduce el número de la opción deseada y sigue las instrucciones en pantalla.

Descripción de las opciones del menú

1. Ver stats

Muestra un resumen completo del estado actual de la civilización: niveles de tecnología, cantidad de edificios, tamaño de cada grupo de unidades, recursos acumulados y generación por minuto.

2. Construir edificios

Permite construir uno de los cinco tipos de edificios: Granja (aumenta comida), Carpintería (madera), Herrería (hierro), Torre Mágica (maná) o Iglesia (necesaria para sacerdotes). Cada edificio tiene un coste en recursos. Si no tienes suficientes, se mostrará un mensaje de error.

3. Mejorar tecnologías

Puedes subir de nivel la tecnología de ataque o la de defensa. El coste aumenta con cada nivel. Las nuevas unidades que crees se beneficiarán de las mejoras.

4. Crear unidades

Elige el tipo de unidad (Swordsman, Spearman, Crossbow, Cannon, ArrowTower, Catapult, RocketLauncherTower, Magician o Priest) y la cantidad. El juego creará tantas como permitan tus recursos. Si no tienes los edificios necesarios (Torre Mágica para magos, Iglesia para sacerdotes), se te informará.

5. Ver amenaza enemiga

Si hay un ejército enemigo aproximándose, te mostrará su composición. Si no, indicará que no hay amenaza.

6. Ver historial de batallas

Muestra los informes resumidos de todas las batallas libradas. Puedes elegir ver el desarrollo detallado de cada una (ataque por ataque).

7. Salir

Cierra el programa.

Ejemplo de una partida

1. Nada más empezar, espera un minuto para acumular recursos iniciales.
2. Construye una Granja y una Herrería.
3. Mejora la tecnología de ataque al nivel 1.
4. Crea 10 Swordsman y 5 Spearman.
5. Espera a que el ejército enemigo ataque (cada 3 minutos).
6. Tras la batalla, consulta el informe.
7. Con los residuos obtenidos, construye más edificios y sigue ampliando tu ejército.

7. Reflexión y mejoras

Dificultades encontradas

1. Mecánica de batalla compleja

La implementación del algoritmo de combate requirió un cuidadoso manejo de las probabilidades, los conteos de unidades por tipo y la generación de residuos. La recursividad en los ataques repetidos (`attackAgain`) obligó a controlar el desbordamiento de pila.

2. Manejo de tiempos con `TimerTask`

Coordinar las tareas de generación de recursos y creación de ejércitos enemigos sin bloquear el menú de consola fue un desafío. Se optó por usar `Timer` con tareas periódicas, lo que permitió una ejecución asíncrona sencilla.

3. Estructura del ejército

La especificación del PDF exigía un array de 9 `ArrayList`. En Java, los arrays de genéricos generan warnings. Se resolvió encapsulando el array en un `ArrayList` de `ArrayList` y proporcionando un método que devuelve la representación en array solo cuando es necesario.

4. Santificación de unidades

Aunque el concepto está definido en el PDF, la implementación completa de la santificación (activación mientras hay sacerdotes y maná, mejora de stats, desactivación al morir el último sacerdote) se simplificó para no sobrecargar la lógica de batalla. Se dejó como mejora futura.

Cómo se superaron

- Pruebas incrementales: Cada nuevo método se probaba individualmente antes de integrarlo.
- Refactorización constante: Cuando se detectaban redundancias o malos olores, se reestructuraba el código (ej. cambio del array de `ArrayList`).
- Uso de logs detallados: El `battleDevelopment` resultó esencial para depurar la mecánica de combate.

Mejoras futuras sugeridas en el PDF

1. Interfaz gráfica (GUI) – El documento original menciona una aplicación gráfica. Podría implementarse con JavaFX o Swing.
 2. Santificación completa – Limitar el número de unidades santificadas por sacerdote y consumir maná periódicamente.
 3. Más tipos de enemigos – Añadir unidades especiales enemigas (magos, sacerdotes) para mayor variedad táctica.
 4. Modo multijugador – Permitir que dos jugadores controlen civilizaciones opuestas.
 5. Persistencia completa – Guardar y cargar el estado del juego desde la base de datos.
-